Towards Efficient Discrete Controller Synthesis: Semantics-Aware Stepwise Policy Design via LLM

Yusei Ishimizu[1], Jialong Li[2*], Takuto Yamauchi[2], Sinan Chen[3], Jinyu Cai[2], Takanori Hirano[2], Kenji Tei[1]
[1] School of Computing, Tokyo Institute of Technology, Tokyo, Japan
[2] Department of Computer Science and Engineering, Waseda University, Tokyo, Japan
[3] Center of Mathematical and Data Sciences, Kobe University, Kobe, Japan
*Corresponding Author: lijialong@fuji.waseda.jp

2024 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia) | 979-8-3315-3083-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCE-ASIA63397.2024.10773792

***Abstract*—Discrete Controller Synthesis (DCS) is an automated generation technique to generate controllers from environment models and requirements. The time and space complexities of DCS are critical because they directly govern the feasibility of using DCS for real-time generation of controllers to cope with environmental changes. Stepwise DCS is considered a promising approach to prevent the construction of ineffective state spaces in the subsequent analysis of the full controller by analyzing sub-controllers that meet part of the requirements at an early stage. However, constructing a stepwise policy is a challenge, as the current methods often require domain experts to manually build it by understanding and reasoning the semantics of each sub-environment model and sub-requirement. To reduce the effort of manual construction, this paper introduces a method that leverages large language models (LLMs) to automate the generation of stepwise policies by utilizing the LLM's ability to understand and reason about the semantics of state machines. The evaluation using GPT-4 on a drone control scenario demonstrates the effectiveness of our method, with an average reduction of 82.2% in states and 91.2% in transitions.**

***Index Terms*—Discrete Controller Synthesis, Stepwise Policy, Large Language Model**

I. Introduction

Discrete Controller Synthesis (DCS) is an automated generation technique to generate controllers, which is also referred to as "action plan" or "system specification" in some studies. As the generated controller can formally guarantee the satisfaction of requirements under the given environment, DCS is regarded as a valuable automation technique for enhancing the reliability of discrete event systems. Recently, DCS has been widely used to re-synthesize the controller at runtime [1], enabling the system's adaptability against environmental changes at runtime. Such use of DCS has also been extensively explored in consumer electronics, with applications including service mobile robots [2], drone missions [3], smart home [4], and medical management [5].

In this context, the computation time and memory required for DCS are crucial, as the former determines the system's response speed to environmental changes, while the latter decides its applicability in resource-constrained systems, such as embedded systems. To enhance DCS performance, numerous studies have adopted various approaches. These include localized analysis specifically tailored to particular environmental changes [6], search heuristics within the state space [7], and changing the granularity of modeling [8]. Furthermore, a technique known as stepwise DCS (SDCS) has been explored, which aims to analyze sub-controllers that meet part of the requirements at an early stage to avoid the construction of requirement-violated state spaces in the subsequent analysis of the full controller [9]. This approach is deemed promising as it facilitates state reduction during the construction phase of the state space, rather than during the search phase, and is expected to synergize with other approaches.

However, a challenge with employing SDCS is defining an effective stepwise policy, which dictates the order in which sub-controllers are analyzed. The choice of stepwise policy is critical as it directly influences the effectiveness of SDCS over traditional DCS methods. Specifically, the purpose of the stepwise policy is to maximize the compression of the state space in the earlier steps, but it is often uneconomical to formally analyze the optimal stepwise policy due to the combinatorial explosion when matching sub-environment models to sub-requirements [9]. In other words, formally analyzing the optimal policy design through the model structure would incur a higher computational cost than DCS itself. Consequently, human experts are typically required to manually construct such a stepwise policy based on their understanding of the semantic meanings inherent in each sub-environment and sub-requirement model's internal state spaces.

To alleviate the need for extensive manual effort, this paper proposes an approach that utilizes large language models (LLMs) to automatically generate stepwise policies. The rationale for using LLMs is their capability to comprehend the environment and requirements represented as state machines and to reason about their internal semantics [10]. This aligns with the traditional strategies of human experts manually crafting stepwise policies. The contributions of this paper include:

- We explicitly define the process, constraints, and expert insights necessary for constructing stepwise policies, and embed such context to the prompt design, thereby facilitating the use of LLMs;
- We validate the effectiveness of our LLM-based approach through experiments conducted with a drone control scenario.
- We have made our experiment reproduction kit open

Authorized licensed use limited to: Waseda University. Downloaded on March 12,2026 at 12:23:34 UTC from IEEE Xplore. Restrictions apply.

source, which includes prompts and experiment models. https://github.com/Uuusay1432/LLM4SDCS.

The remainder of the paper is organized as follows: Section II provides the necessary background. Section III introduces a drone scenario as the running example. Section IV details the proposed approach. Section V presents the evaluation of our approach. Finally, Section VI concludes the paper and suggests future directions.

II. Background

A. Model Formalization

This section outlines the formalization of the models involved in DCS, as depicted in Fig. 1.

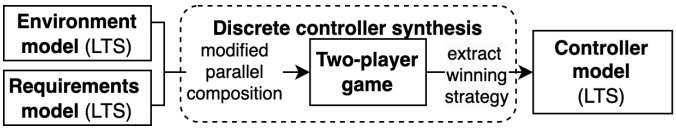

Fig. 1: Overview of DCS

Definition 1: *(Labelled Transition Systems, LTS) An LTS is* $D = (S, L, \Delta, s_0)$ *where* S *is a finite set of states,* $L = L_C \cup L_M$ *with* L_C *and* L_M *being disjoint sets of controllable and monitorable labels respectively,* Δ *is a transition relation* $S \times L \times S$*, and* $s_0 \in S$ *is the initial state.*

LTS is employed to model the environment, requirements, and controllers. The requirement model (for safety properties), is a specialized form of LTS, which includes an additional set $S_E \subset S$, representing error states where requirements are violated.

Definition 2: *(Two-player Game) A two-player game (hereafter game) is* $SG = (S_{sg}, L, \Gamma, s_{sg_0}, X)$*, where* S_{sg} *is a finite set of states divided into* $S_C \cup S_M$*,* $L = L_C \cup L_M$ *is a set of labels,* Γ *includes monitorable (*Γ^-*) and controllable (*Γ^+*) transition relations,* $s_{sg_0} \in S_{sg}$ *is the initial state, and* X *denotes states where the requirement is violated.*

A game is derived from the parallel composition [11] of environment and requirement models. The two players in the game are (1) the system that selects controllable transitions to execute and (2) the operational environment that determines which monitorable transitions to happen. Specifically, the process of synthesizing the controller essentially involves identifying the state space that could violate the requirements and then extracting the remaining state space that can guarantee the requirements from the game.

B. Stepwise Discrete Controller Synthesis

Stepwise Discrete Controller Synthesis (SDCS) is a method of DCS for partial and step-by-step analysis of the (sub-)controller, focusing on the relationship between the environment model and the requirement model.

The algorithm of SDCS is illustrated in Fig.2. Suppose we are given a set of environment models E and a set of requirement models R. Initially, the requirement model r^* to be analyzed is selected by a Stepwise Policy, and E^*, defined as the set of environment models to be synthesized with r^*, is derived. Subsequently, executing DCS between r^* and E^* results in the creation of a new environmental model e^* under which the requirements of r^* are met. Finally, the union of E not included in E^* and e^* is redefined as the new E, and R not included in r^* is redefined as the new R. By repeating this process until R is depleted, a controller that satisfies all requirements in R can be generated.

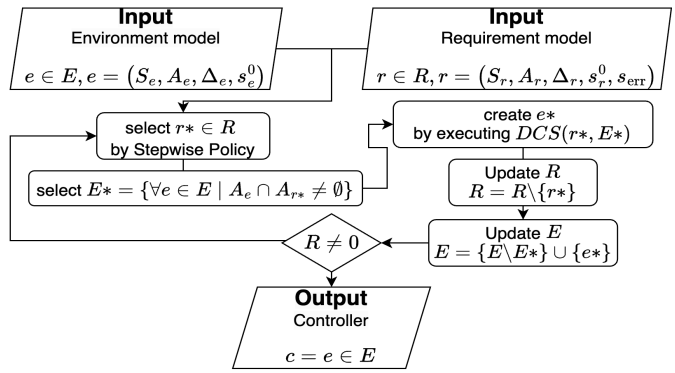

Fig. 2: Algorithm of SDCS.

SDCS has the potential to reduce computational space; however, the effectiveness of this reduction is contingent upon the stepwise policy that selects the appropriate requirement model for DCS application. It is important to note that an inappropriate stepwise policy could potentially increase computational costs. This issue will be introduced by an example in Sections III and V.

III. Running Example of Drone Scenario

This section demonstrates the impact of stepwise policy on the performance of SDCS using a drone mission scenario [3]. For ease of presentation, we assume the drones operate within a one-dimensional altitude.

The system comprises two environment models: "ALTITUDE" indicates the drone's height (0-4), and "BATTERY" displays the remaining battery power (low or high), as depicted in Fig. 3(a). The drone could execute the following actions: `takeoff` (height from 0 to 1), `land`, `up.n`, `down.n`, and `battery.X` (battery status change to X).

The requirements for the system include: ensuring no altitude increase when the BATTERY level is low, termed NOT_UP_BL (Fig. 3(b)-(i)), and restricting altitude to a maximum of 3 (e.g., legal restrictions), termed NOT_UP_3 (Fig. 3(b)-(ii)).

The stepwise policy is critically important in SDS as it directly influences the scale of the analysis space and indirectly affects computation time. We illustrate this with two examples. The first stepwise policy: if the NOT_UP_BL model is synthesized first, the relevant environment models are ALTITUDE and BATTERY (see Fig. 3(c)). Subsequently, the NOT_UP_3 model is synthesized incrementally, and the final controller is depicted in Fig. 3(e). Alternatively, under

Authorized licensed use limited to: Waseda University. Downloaded on March 12,2026 at 12:23:34 UTC from IEEE Xplore. Restrictions apply.

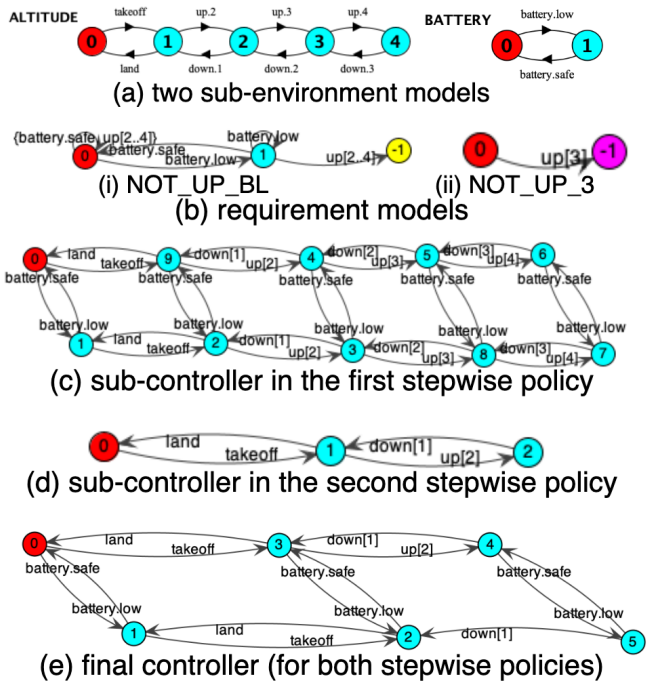

Fig. 3: Comparison of stepwise policy using drone scenario.

the second stepwise policy: if NOT_UP_3 is synthesized first, the initially synthesized model is shown in Fig. 3(d), with the final controller identical to Fig. 3(e). Comparing these two policies, we observe that the first policy requires analyzing 8 states (sub-controller) and 4 states (final controller), whereas the second policy only involves analyzing 2 states and 4 states, respectively.

IV. Proposal

Due to space limitations, this section briefly introduces the outline of our method and two critical expert insights utilized within it. Interested readers may refer to our open-source detailed prompt designs. It is important to note that these insights presuppose a thorough understanding of the model and its internal semantics, hence we refer to this as "Semantics-Aware Stepwise Policy Design".

A. Overview

In this paper, we use LLMs to simulate the design processes of human domain experts. Specifically, the prompts to the LLMs include: (i) background knowledge, comprising definitions of DCS, SDCS, and LTS; (ii) the task definition to design a stepwise policy; (iii) domain expert experience and knowledge. We consulted three experts in the DCS domain and interviewed them for their experiences and insights in designing stepwise policies. By embedding the aforementioned knowledge in prompts, LLMs are enabled to semantically reason and design stepwise policies akin to human experts.

B. Prioritizing Requirements with Fewer Monitored Models

This insight emphasizes the strategy of initiating synthesis from requirement models that impact fewer sub-environment models to achieve more efficient results. Specifically, for each requirement, there exists sub-environment model(s) that possess the actions required by the requirement. We refer to the set of sub-environment models corresponding to a requirement model as the monitored model, which semantically indicates whether a requirement impacts the sub-environments/functionalities or not. In this context, the first expert insight is that it is advisable to initiate synthesis from requirement models associated with fewer monitored models, as early reductions in smaller-scale models can better reflect subsequent synthesis steps.

In the drone scenario, the monitored model for the requirement NOT_UP_3 consists of one ALTITUDE, while the monitored model for the requirement NOT_UP_BL comprises both ALTITUDE and BATTERY. Therefore, in this scenario, it is preferable to begin synthesis with the requirement NOT_UP_3.

C. Prioritizing Requirements with More State Reduction

This insight highlights the approach of determining the synthesis order by assessing which requirements can eliminate more violation states within the environment model, thereby optimizing the synthesis process. Specifically, DCS requires identifying states in the environment model that violate requirements and eliminating these violation states. Thus, the second expert insight is that, for the same environment model, if requirement R1 can eliminate more violation states than R2, synthesis should commence with R1.

To illustrate the concept with ease of understanding, we present an extreme example. We introduce an additional requirement, NOT_UP_2, which mandates "Do not exceed the limited altitude 2." In the monitored model ALTITUDE of both NOT_UP_3 and NOT_UP_2, the potential reduction in the number of states is expected to be greater for NOT_UP_2 due to its stricter constraints. Therefore, synthesis should commence with NOT_UP_2.

It should be noted that these two insights are not orthogonal; that is, the insights may conflict under certain conditions. Moreover, these insights are merely heuristics in design strategy; the corresponding stepwise policy does not guarantee optimality.

V. Preliminary Evaluation

Our evaluation aims to address the following research question: How significant is the reduction in the number of states and transitions within the analysis space?

A. Experiment Setting

We conduct an experiment utilizing a drone mission scenario, which is a three-dimensional extension of the previously described drone model. The experiment is performed using MTSA (Modal Transition System Analyser) [12] on a MacBook equipped with an M1 chip and 16GB of memory. For the experiment, we employ OpenAI's GPT-4 and GPT-3.5, which are the two most representative LLMs. For each LLM, we conduct five experiments and take the average.

Authorized licensed use limited to: Waseda University. Downloaded on March 12,2026 at 12:23:34 UTC from IEEE Xplore. Restrictions apply.

B. Experiment Results

The experimental results are summarized in Table I. The table compares the performance of a non-stepwise baseline (DCS) with SDCS employing various stepwise policies. These stepwise policies include: (i) Optimum: the best solution identified by trialing all possible stepwise orders; (ii) GPT-4; (iii) GPT-3.5; and (iv) Random: a randomly generated synthesis order. For (ii), (iii), and (iv), we also record the number of times each generated policy was optimal.

TABLE I: Experiment Results

Method	States	State Reduction	Transition	Transition Reduction	Optimum Rate
DCS	4,002	-	26,204	-	-
Optimum	546	88.6%	2,296	91.2%	-
GPT-4	709	82.2%	3,975	84.8%	4/5
GPT-3.5	3,371	15.8%	20,449	22.0%	1/5
Random	3,311	17.3%	21,562	17.7%	1/5

As shown, SDCS designed with GPT-4 achieved substantial reductions in both states and transitions when compared to the traditional DCS method. Specifically, the number of states was reduced from 4,002 to 709, marking an 82.2% reduction, while transitions decreased by 84.8%, from 26,204 to 3,975. In contrast, GPT-3.5 and a randomly generated stepwise policy showed lesser improvements. GPT-3.5 and Random managed a 15.8% and 17.3% reduction in states, and a 22.0% and 17.7% reduction in transitions, respectively.

Compared with the optimal policy, which achieves reductions of 88.6% and 91.2% in states and transitions, respectively. Over five experimental runs, GPT-4 achieved this optimum in four instances, indicating a relatively high efficacy and robustness. In contrast, GPT-3.5 and Random reached the optimal state reduction in only one out of five trials.

C. Discussion and Limitations

Through the aforementioned experiments, we believe we have preliminarily validated the capability of LLMs to design stepwise policies through the understanding of model semantics. Notably, the superior performance of GPT-4 compared to GPT-3.5 is an expected outcome, as comprehending model semantics and basing decisions on two expert insights fundamentally constitute a complex reasoning problem.

Additionally, it have highlighted another advantage of LLM-based automated design over manual design: LLMs can utilize the runtime contexts (i.e., models that are updated at runtime) to make decisions. Conversely, humans typically must design all-order steps in advance during the design phase, rather than during runtime resynthesis, as this would require human experts to be on call 24/7 and capable of making decisions within seconds like LLMs. This characteristic is crucial as changes in the environment model due to environmental variations can lead to changes in semantics, thereby necessitating potential modifications to the stepwise policy.

Of course, we acknowledge that our evaluation and approaches are still preliminary. As future research, more detailed evaluations are required, including larger-scale systems, more diverse domain applications, and more complex design decisions. Additionally, for the proposals, employing more advanced prompt techniques such as multi-LLM discussions could further enhance performance.

VI. Conclusion and Future Work

This paper employs LLMs to automatically generate stepwise policies for SDCS, significantly reducing the need for manual design effort. To enhance the performance of LLMs in this task, we have embedded expert knowledge of semantic considerations into the prompts. Evaluations conducted in a drone control scenario confirm that our approach can effectively generate stepwise policies that reduce the states by 82.2% and transitions by 91.2%.

Future research will focus on several key directions. Firstly, we plan to do a more detailed evaluation using more large-scale applications. Secondly, we plan to interview industrial users of DCS to gather more detailed design know-how to enhance our prompt design. Lastly, we intend to further explore the application of LLMs' semantic understanding capabilities in DCS, such as the automatic correction of environment or requirement models when specifications cannot be synthesized.

References

[1] T. Buckworth, D. Alrajeh, J. Kramer, and S. Uchitel, "Adapting specifications for reactive controllers," in *2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2023, pp. 1–12.

[2] J. Li, T. Yamauchi, N. Li, Z. Chen, M. Zhang, T. Hirano, and K. Tei, "Demonstration of a real-world self-adaptive robot path-finding using discrete controller synthesis," in *2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, 2023, pp. 27–28.

[3] S. A. Zudaire, L. Nahabedian, and S. Uchitel, "Assured mission adaptation of uavs," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 16, no. 3–4, jul 2022.

[4] J. Li, W. Manzano, T. Yamauchi, N. Matsuyama, E. Y. Nakagawa, and K. Tei, "Employing discrete controller synthesis for developing systems-of-systems controllers," in *12th ACM/IEEE International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems (SESoS@ICSE)*, April 2024.

[5] L. Nahabedian, V. Braberman, N. D'ippolito, J. Kramer, and S. Uchitel, "Dynamic reconfiguration of business processes," in *Business Process Management*, 2019, pp. 35–51.

[6] J. Li, K. Aizawa, K. Tei, and S. Honiden, "Efficient difference analysis algorithm for runtime requirement degradation under system functional fault," in *IEEE 18th International Conference on Embedded and Ubiquitous Computing (EUC)*, 2020, pp. 33–40.

[7] T. Delgado, M. S. Sorondo, V. Braberman, and S. Uchitel, "Exploration policies for on-the-fly controller synthesis: a reinforcement learning approach," in *Thirty-Third International Conference on Automated Planning and Scheduling (ICAPS)*, 2023.

[8] J. Li and K. Tei, "Done is better than perfect: Iterative adaptation via multi-grained requirement relaxation," in *2022 IEEE 30th International Requirements Engineering Conference (RE)*, 2022, pp. 288–294.

[9] T. Yamauchi and K. Tei, "An analysis space reduction for discrete controller synthesis by stepwise partial synthesis," *IEICE Transactions D*, vol. J106-D, no. 4, pp. 218–230, 2023.

[10] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, and K. Tei, "Generative ai for self-adaptive systems: State of the art and research roadmap," *ACM Trans. Auton. Adapt. Syst.*, 2024.

[11] J. Magee and J. Kramer, *Concurrency: State Models and Java Programs*, 2nd ed. Wiley Publishing, 2006.

[12] N. D'Ippolito, D. Fischbein, M. Chechik, and S. Uchitel, "Mtsa: The modal transition system analyser," in *23rd IEEE/ACM International Conference on Automated Software Engineering*, 2008, pp. 475–476.

Authorized licensed use limited to: Waseda University. Downloaded on March 12,2026 at 12:23:34 UTC from IEEE Xplore. Restrictions apply.